

```
public class ViewHolder extends RecyclerView.ViewHolder
{
    public UserViewBinding mBinding;

    public ViewHolder(UserViewBinding viewBinding) {
        super(viewBinding.getRoot());
        mBinding = viewBinding;
    }
}
```

Now, when you create a ViewHolder...

```
@Override
public ViewHolder onCreateViewHolder(ViewGroup parent,
int viewType) {
    return new ViewHolder(UserViewBinding.
        inflate(LayoutInflater.from(parent.getContext()), parent,
        false));
}
```

...you may be wondering where the inflate method came from. Note that when our binding classes are generated, they are given these convenient static methods for easy initialisation. Now, in *onBindViewHolder*, you just set the variable.

```
@Override
public void onBindViewHolder(UserViewHolder holder, int
position) {
    holder.mBinding.setUser(mUserDatas.get(position));
    holder.mBinding.executePendingBindings();
}
```

And with almost this much code, you can come up with something like the screenshot in Figure 1. Be sure to *executePendingBindings()*; otherwise, *RecyclerView* would do two layout passes.

Binding adapters

Binding adapters are one of the coolest things that the data binding framework can do. If you want to override the functionality of what an attribute does or want to introduce your own XML attribute to do some work, binding adapters can be of help. For instance, if you want to set an image in your *ImageView* in XML, but the image is a URL for a resource on your server, the URL of the image is stored in the model you set on your layout file. So binding adapters can help you here.

```
@BindingAdapter("imageUrl")
public static void setImage(ImageView image, String url) {
    Glide.with(image.getContext()).load(url).crossFade().
    centerCrop().into(image);
}
```

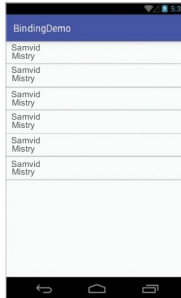


Figure 1: RecyclerView with data binding

You can create binding adapter methods by annotating them with the *@BindingAdapter* method you pass in the attribute you want to override. In method parameters, the first parameter should be the type of view you want this annotation to be on, and the other should be the type of value in the parameter that you set in the annotation. Then, inside the method, you can do anything with the parameter

and the view. This is how a simple binding adapter can be written. If you want to do something a little bit more complex, like passing multiple parameters in binding adapters, that can also be done, as follows:

```
@BindingAdapter(value = {"imageUrl", "placeholder"},
requireAll = false)
public static void setImage(ImageView image, String url,
Drawable placeholder) {
    Glide.with(image.getContext()).load(url).
    placeholder(placeholder).crossFade().centerCrop().
    into(image);
}
```

You just need to take care that the ordering of the values for attributes and the value types match the types of parameters; otherwise, your method may not be called. 

References

- [1] <https://developer.android.com/topic/libraries/data-binding/index.html>
- [2] <https://github.com/google/android-ui-toolkit-demos/tree/master/DataBinding/DataBoundRecyclerView>
- [3] <https://www.youtube.com/watch?v=DAmMN7m3wLU>
- [4] <https://github.com/samvidmistry/data-binding-demo> - Link to demo used in article

By: Prof. Prakash Patel and Samvid Mistry

Prof. Prakash Patel is working as assistant professor in the computer engineering department at Nirma University. He can be contacted at prakashpatel@nirma.ac.in

Samvid Mistry is an Android developer at SculptSoft. He holds a diploma in computer engineering from Nirma University. He is available at mistrysamvid@gmail.com, <http://www.samvidinfotech.in>